# TD N° 4

## Fragmentation

La fragmentation permet de diviser une table en plusieurs sous-tables telles que la reconstruction de la table originale est toujours possible. Il y a deux types de fragmentation : **horizontale** et **verticale**.

## Fragmentation Horizontale

On a deux types de fragmentation horizontale :

- **FHP (Fragmentation Horizontale Principale)** : on divise la table en utilisant l'opérateur de restriction $\sigma$ :

$$\sigma_{\text{condition}}(\text{Table})$$

- **FHD (Fragmentation Horizontale Dérivée)** : on divise la table en utilisant l'opérateur de semi-jointure $\ltimes$ entre la table qui possède une clé étrangère faisant référence à une **FHP** :

$$\text{Table} \ltimes \text{FHP}_{\text{référence}} = \text{Table} \bowtie \left(\prod_{\text{clé_primaire}}(\text{FHP})\right)$$

L'opérateur de reconstruction est l'union $\bigcup$ :

$$\text{Table} = \bigcup_{i=1}^{n} \text{Frag}_i$$

**Fragmentation Horizontal**

```
-- FHP
CREATE TABLE TABLE_NAME (
    ...
 )
 PARTITION BY LIST (ATTRIBUTE_NAME) (
    PARTITION FRAG1_NAME VALUES (VALUES_1),
    PARTITION FRAG2_NAME VALUES (VALUES_2),
    ....
    PARTITON FRAGN_NAME VALUES(DEFAULT) -- optional to use default means rest of values
 );

-- FHD
CREATE TABLE TABLE_NAME (
    ...,
CONSTRAINT CONSTRAINT_NAME foreign key(TABLE_NAME_FK) REFERENCES
REFERENCED_TABLE(REFERENCES_TABLE_PK) ) PARTITION BY REFERENCE (CONSTRAINT_NAME);
```

## Fragmentation Verticale

On utilise l'opérateur de projection $\prod$ :

$$\prod_{\text{attribut}_1,\ldots,\text{attribut}_m}(\text{Table})$$

**Attention :** il faut toujours inclure la clé primaire dans les attributs de la projection afin de pouvoir faire la reconstruction avec l'opérateur de jointure $\bowtie$ :

$$\text{Table} = \bigbowtie_{i=1}^{n} \text{Frag}_i$$

**Fragmentation Vertical**

```
CREATE TABLE FRAG_NAME AS
SELECT ATT_1 , ... , ATT_M FROM TABLE_NAME;
```

## Arbre Algebrique

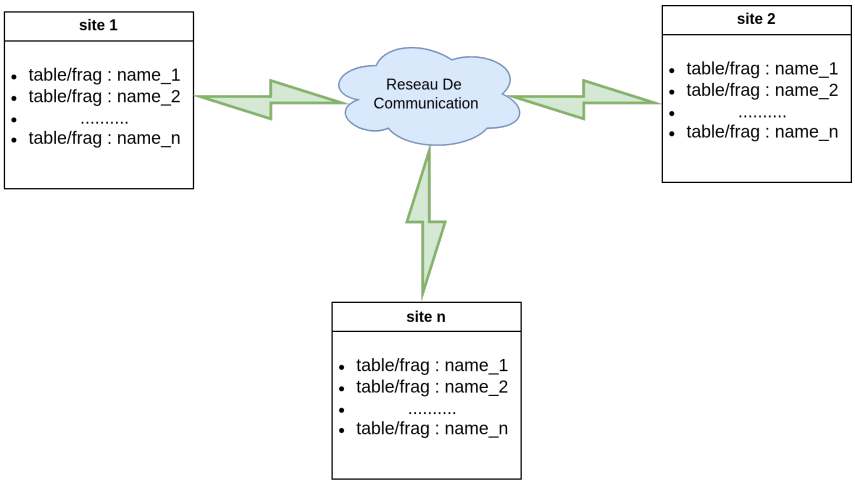
1. **Arbre Algebrique Global** :On utilise les tables d'origins.
2. **Arbre Algebrique Canonique** :on fait que remplacer les table avec leur fragments et utiliser l'operateur de reconstruction $\bigcup$ si horizontal , sinon si vertical $\bowtie$
3. **Arbre Simplifie** :
   - **Regle d'optimisation Global** :descent des operateurs de reduction de taille (projection,restriction) avant les operateurs $\bigcup$ et $\bowtie$
   - **Distribution Du Join Par Rapport A L'union** :

     $$A \bowtie (B \bigcup C) = (A \bowtie B) \bigcup (A \bowtie C)$$

   - **Supression Des Restrictions, Projection Inutiles**.
   - **Supression Des Fragments Vides**.

## Schéma d'allocation

Un schéma qui définit tout les sites avec leur fragments et tables de la base de données distribuée :




## Stratégie

On a 3 cas global :

- Envoie tout les fragments/tables vers d'autre site et traiter la requete.
- Envoie les fragments/tables apres restriction/projection/jointure.
- Envoie des tuples de tout le fragments/table vers d'autre site qui ensuite vont repondre avec un bit.
- Envoie des tuples de tout le fragments/table apres restriction/projection/jointure vers d'autre site qui ensuite vont repondre avec un bit.

## Temps

$$\text{temp}_{envoie-table} = \text{delai-access} + \frac{\text{nb}_{\text{tuple}} \cdot \text{taille}_{\text{tuple}}}{\text{taux de transmission}}$$

$$\text{temp}_{envoie-bit} = \text{delai-access} + \frac{\text{taille}_{\text{bit}} = \frac{1}{8}}{\text{taux de transmission}}$$

# Exercice 1

Fragmentation de production :

Puisque l'entreprise a quatre sites et que chaque site est responsable de la production de différents composants, on va effectuer une fragmentation horizontale principale sur l'attribut composant.

$$\begin{aligned}
\text{Production}_{\text{UC}} &= \sigma_{\text{Composant = 'Unite Centrale'}}(\text{Production}) \\
\text{Production}_{\text{Clavier}} &= \sigma_{\text{Composant = 'Clavier'}}(\text{Production}) \\
\text{Production}_{\text{Ecran}} &= \sigma_{\text{Composant = 'Ecran'}}(\text{Production}) \\
\text{Production}_{\text{Cable}} &= \sigma_{\text{Composant = 'Cable'}}(\text{Production})
\end{aligned}$$

Fragmentation de vente :

Puisque vente possède une clé étrangère `numserie` qui fait référence à la table production, déjà fragmentée en **FHP**, on va effectuer une fragmentation horizontale dérivée.

$$\begin{aligned}
\text{Vente}_{\text{UC}} &= \text{Vente} \ltimes \text{Production}_{\text{UC}} \\
\text{Vente}_{\text{Clavier}} &= \text{Vente} \ltimes \text{Production}_{\text{Clavier}} \\
\text{Vente}_{\text{Ecran}} &= \text{Vente} \ltimes \text{Production}_{\text{Ecran}} \\
\text{Vente}_{\text{Cable}} &= \text{Vente} \ltimes \text{Production}_{\text{Cable}}
\end{aligned}$$

Fragmentation de client :

On a les clients d'Alger et de Sétif. ainsi que ceux d'Annaba et Tarf qui s'adressent au point de vente d'Annaba on va donc effectuer une fragmentation horizontale principale sur l'attribut villeClient.

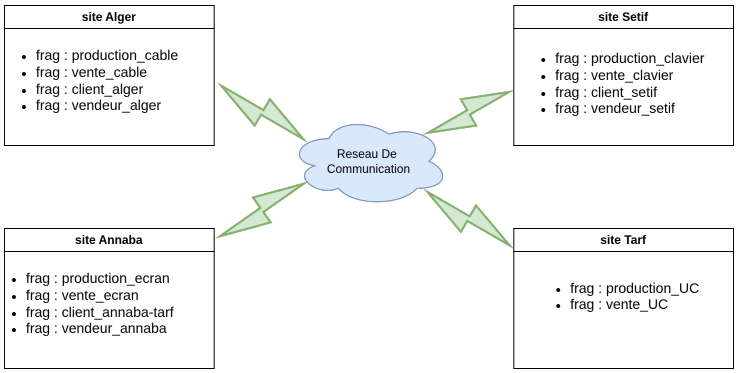
$$\begin{aligned}
\text{Client}_{\text{Alger}} &= \sigma_{\text{villeClient = 'Alger'}}(\text{Client}) \\
\text{Client}_{\text{Setif}} &= \sigma_{\text{villeClient = 'Setif'}}(\text{Client}) \\
\text{Client}_{\text{Annaba-Tarf}} &= \sigma_{\text{villeClient in ('Annaba','Tarf')}}(\text{Client})
\end{aligned}$$

Fragmentation de vendeur :

On a trois poitns de vente Alger , Annaba et Setif on va donc effectuer une fragmentation horizontale principale sur l'attribut villeVendeur.

$$\begin{aligned}
\text{Vendeur}_{\text{Alger}} &= \sigma_{\text{villeVendeur = 'Alger'}}(\text{Vendeur}) \\
\text{Vendeur}_{\text{Setif}} &= \sigma_{\text{villeVendeur = 'Setif'}}(\text{Vendeur}) \\
\text{Vendeur}_{\text{Annaba}} &= \sigma_{\text{villeVendeur ='Annaba'}}(\text{Vendeur})
\end{aligned}$$

Schéma d'allocation :

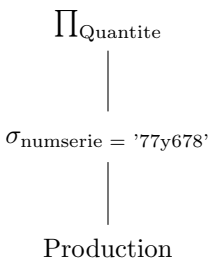



R1 : Quelle est la quantité disponible du produit de numéro de série '77y6878'.

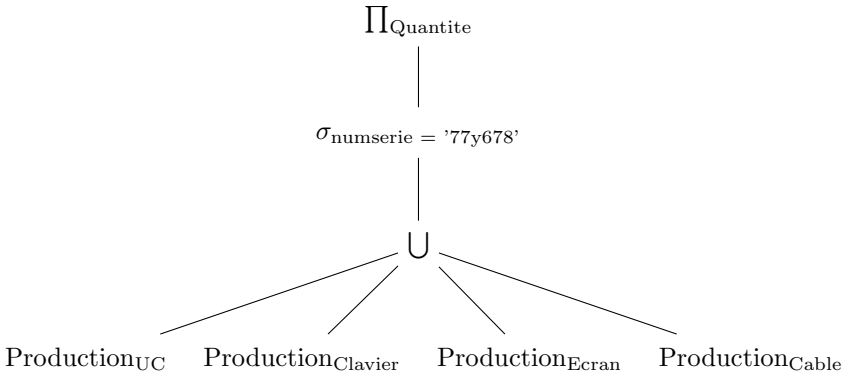
**R1**

```
SELECT QUANTITE FROM PRODUCTION
WHERE NUMSERIE = '77y6878';
```

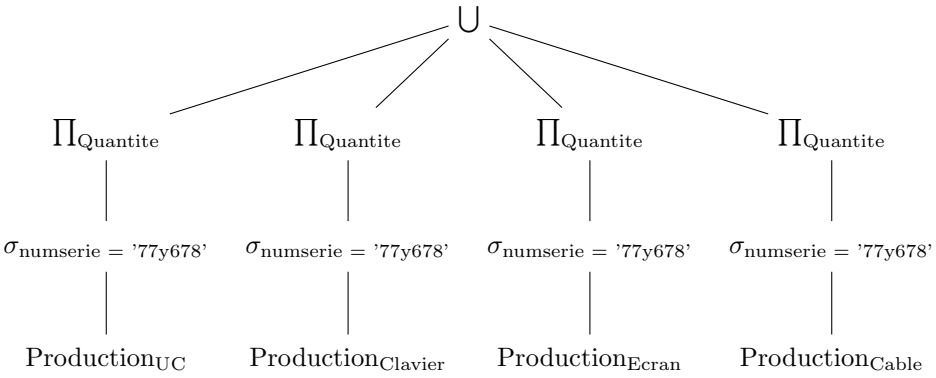
Arbre algebrique global :




Arbre algebrique canonique (On remplace la table avec ces fragments et on utilise l'operateur de reconstruction $\bigcup$ ) :




regle optimisation generale (descente des operateurs de reductions de taille projection et restriction avant $\bigcup$) :

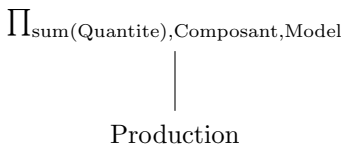


R2 : La somme des quantités de produits par composant et modèle.

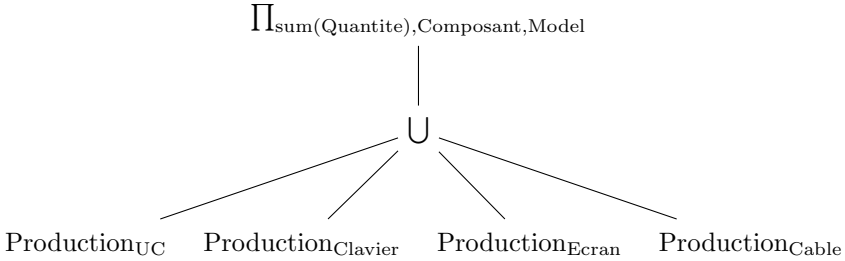
**R2**

```
SELECT SUM(QUANTITE),COMPOSANT,MODEL FROM PRODUCTION
GROUP BY COMPOSANT,MODEL;
```

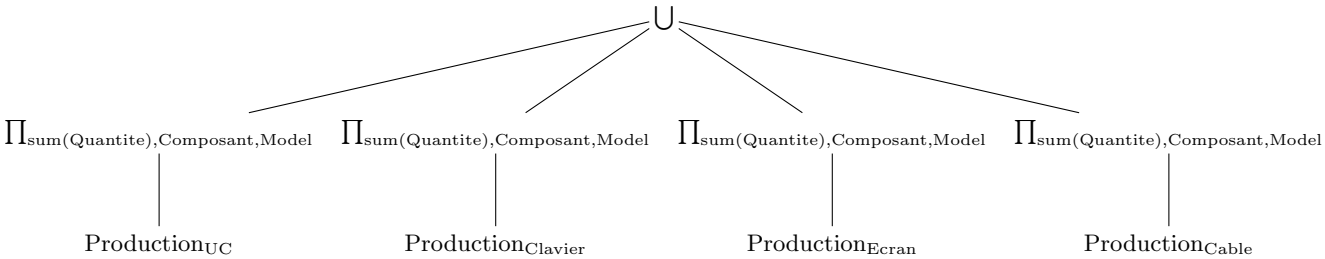
Arbre algebrique global :




Arbre algebrique canonique (On remplace la table avec ces fragments et on utilise l'operateur de reconstruction $\bigcup$ ) :




regle optimisation generale (descente des operateurs de reductions de taille projection avant $\bigcup$) :




- **Stratégie 01** : Envoyer les fragments PROD_UC, PROD_CLAVIER et PROD_CABLE à Annaba et exécuter la requête.
- **Stratégie 02** : Envoyer les projections sur les attributs modèle et quantité pour chaque fragments PROD_UC, PROD_CLAVIER et PROD_CABLE à Annaba et exécuter la requête.
- **Stratégie 03** : Calculer la somme des quantités de produits par composant et modèle pour chaque site et envoyer le résultat à Annaba et exécuter l'union avec le résultat des composants de Annaba.
- **Stratégie 04** : Calculer la somme des quantités de produits par composant et modèle pour les sites Tarf et Sétif puis envoyer le résultat à Alger et exécuter l'union avec la somme des quantités de produits par composant et modèle pour site Alger. Envoyer le résultat à Annaba et faire l'union avec le résultat des composants de Annaba

# Exercice 2

Fragmentation d'agence :

Puisque l'ecole a trois sites Alger, Constantine, Oran on va effectuer une fragmentation horizontale principale sur l'attribut villeAG.

$$\begin{aligned}
\text{Agence}_{\text{Alger}} &= \sigma_{\text{villeAG = 'Alger'}}(\text{Agence}) \\
\text{Agence}_{\text{Const}} &= \sigma_{\text{villeAG = 'Constantine'}}(\text{Agence}) \\
\text{Agence}_{\text{Oran}} &= \sigma_{\text{villeAG = 'Oran'}}(\text{Agence})
\end{aligned}$$

Fragmentation de Personnel-Administrative :

Puisque la table possède une clé étrangère `numAG` qui fait référence à la table agence, déjà fragmentée en **FHP**, on va effectuer une fragmentation horizontale dérivée.

$$\begin{aligned}
\text{P-A}_{\text{Alger}} &= \text{P-A} \ltimes \text{Agence}_{\text{Alger}} \\
\text{P-A}_{\text{Const}} &= \text{P-A} \ltimes \text{Agence}_{\text{Const}} \\
\text{P-A}_{\text{Oran}} &= \text{P-A} \ltimes \text{Agence}_{\text{Oran}}
\end{aligned}$$

Fragmentation de Vehicule:

Puisque la table possède une clé étrangère `numAG` qui fait référence à la table agence, déjà fragmentée en **FHP**, on va effectuer une fragmentation horizontale dérivée.

$$\begin{aligned}
\text{Vehicule}_{\text{Alger}} &= \text{Vehicule} \ltimes \text{Agence}_{\text{Alger}} \\
\text{Vehicule}_{\text{Const}} &= \text{Vehicule} \ltimes \text{Agence}_{\text{Const}} \\
\text{Vehicule}_{\text{Oran}} &= \text{Vehicule} \ltimes \text{Agence}_{\text{Oran}}
\end{aligned}$$

Fragmentation de Client:

Puisque la table possède une clé étrangère `numAG` qui fait référence à la table agence, déjà fragmentée en **FHP**, on va effectuer une fragmentation horizontale dérivée.

$$\begin{aligned}
\text{Client}_{\text{Alger}} &= \text{Client} \ltimes \text{Agence}_{\text{Alger}} \\
\text{Client}_{\text{Const}} &= \text{Client} \ltimes \text{Agence}_{\text{Const}} \\
\text{Client}_{\text{Oran}} &= \text{Client} \ltimes \text{Agence}_{\text{Oran}}
\end{aligned}$$

Fragmentation de Cours:

Puisque la table possède une clé étrangère `numCli` qui fait référence à la table client, déjà fragmentée en **FHP**, on va effectuer une fragmentation horizontale dérivée.

$$\begin{aligned}
\text{Cours}_{\text{Alger}} &= \text{Cours} \ltimes \text{Client}_{\text{Alger}} \\
\text{Cours}_{\text{Const}} &= \text{Cours} \ltimes \text{Client}_{\text{Const}} \\
\text{Cours}_{\text{Oran}} &= \text{Cours} \ltimes \text{Client}_{\text{Oran}}
\end{aligned}$$

Fragmentation d'Examen:

Puisque la table possède une clé étrangère `numCli` qui fait référence à la table client, déjà fragmentée en **FHP**, on va effectuer une fragmentation horizontale dérivée.

$$\begin{aligned}
\text{Examen}_{\text{Alger}} &= \text{Examen} \ltimes \text{Client}_{\text{Alger}} \\
\text{Examen}_{\text{Const}} &= \text{Examen} \ltimes \text{Client}_{\text{Const}} \\
\text{Examen}_{\text{Oran}} &= \text{Examen} \ltimes \text{Client}_{\text{Oran}}
\end{aligned}$$

Autre fragmentation possible pour vehicule et cours :

- vehicule on peut faire une **FHD** avec P-A grace a la cle etrangere `num-Instructeur`.
- Cours on peut faire une **FHD** avec Vehicule grace a la cle etrangere `numV` , ou une **FHD** avec P-A grace a la cle etrangere `num-Instructeur`.

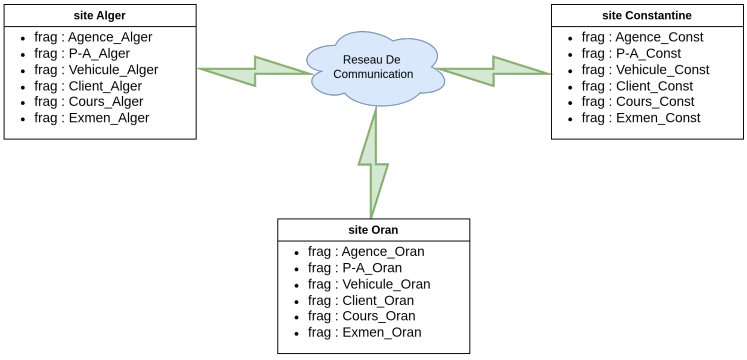
Toute les fragmentations donne le meme resultat 2 fragment Alger Constantine Oran car il sont tous des **FHD** de table qui on ete fragmente de la meme maniere par rapport a la ville.

**Fragmentation**

```oracle
-- FHP
CREATE TABLE AGENCE (
    NUMAG INT,
    VILLEAG VARCHAR2(50),
    NUM-GERANT INT,
    NBR-INSTRUCTEURS INT,
    CONSTRAINT PK_AGENCE PRIMARY KEY (NUMAG),
    CONSTRAINT FK_AGENCE_PA FOREIGN KEY (NUM-GERANT) REFERENCES PA(NUMP)
 )
 PARTITION BY LIST (VILLEAG) (
    PARTITION AGENCE_ALGER VALUES ('ALGER'),
    PARTITION AGENCE_CONST VALUES ('CONSTANTINE'),
    PARTITION AGENCE_ORAN VALUES('ORAN')
 );

-- FHD
CREATE TABLE CLIENT (
    NUMCLI INT,
    NOMCLI VARCHAR2(50),
    ADRESSECLI VARCHAR2(100),
    NUMAG INT,
    AGE INT,
    CONSTRAINT PK_CLIENT PRIMARY KEY (NUMCLI),
    CONSTRAINT FK_CLIENT_AGENCE FOREIGN KEY (NUMAG) REFERENCES AGENCE(NUMAG))
PARTITION BY REFERENCE (FK_CLIENT_AGENCE);
```

Schéma d'allocation :



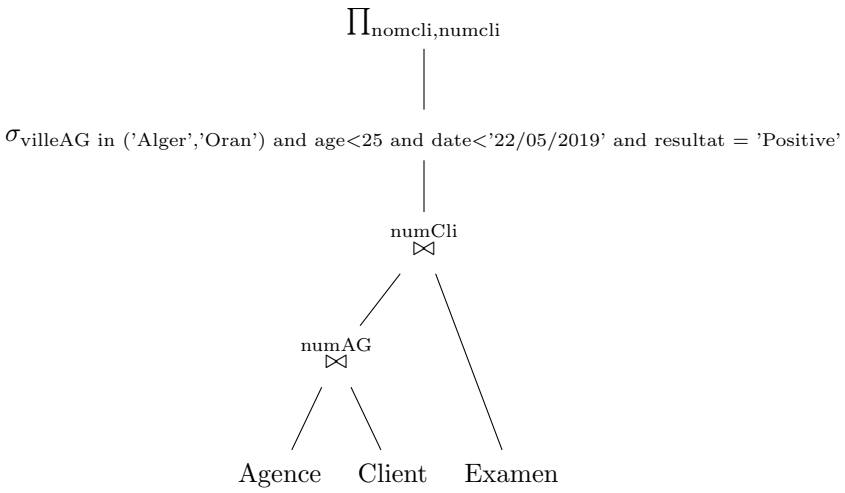

On met une copie du fragment $\text{vehicule}_{\text{Alger}}$ dans le site Oran mais il foudra synchronizer le fragment entre alger and oran quand il ya un changement.

R : Donner les nom et numéro de tous les clients de moins de 25 ans, relevant des agences d'Alger ou d'Oran, qui ont eu un résultat positif à l'examen du 22/05/2019

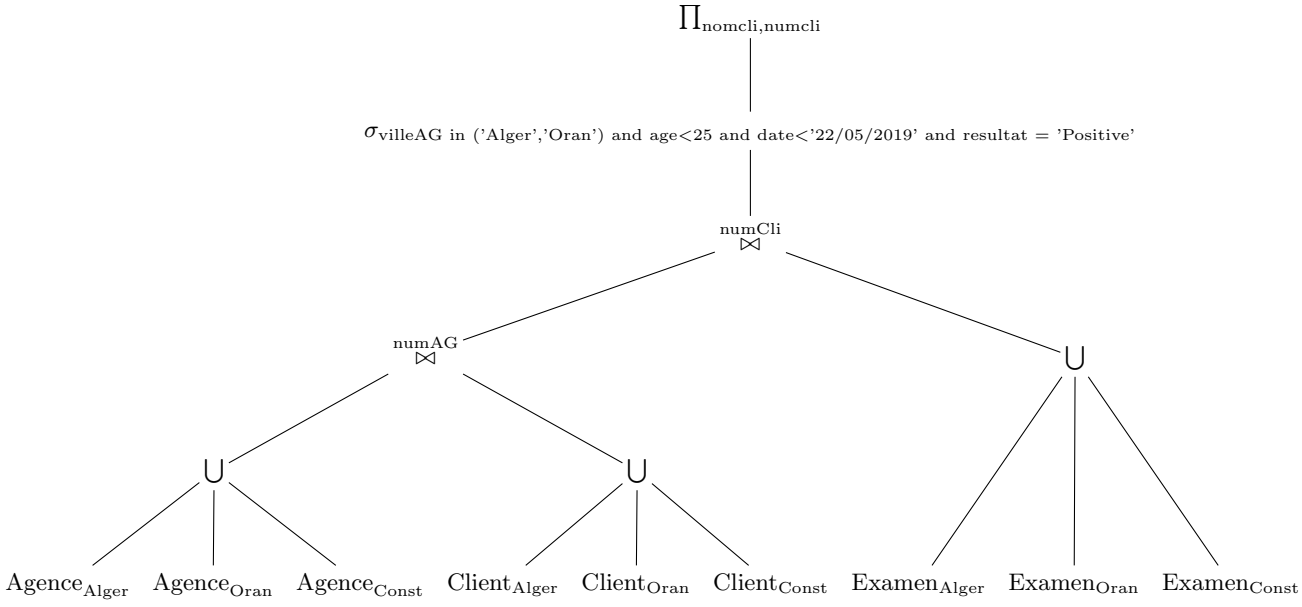
**R**

```
SELECT C.NOMCLI , C.NUMCLI FROM CLIENT C, AGENCE A, EXAMEN E
WHERE A.VILLEAG IN ('Alger','Oran') AND C.AGE < 25 AND E.DATE-EXAMEN = '22/05/2019' AND
E.RESULTAT='POSITIVE' AND C.NUMAG = A.NUMAG AND C.NUMCLI = E.NUMCLI;
```

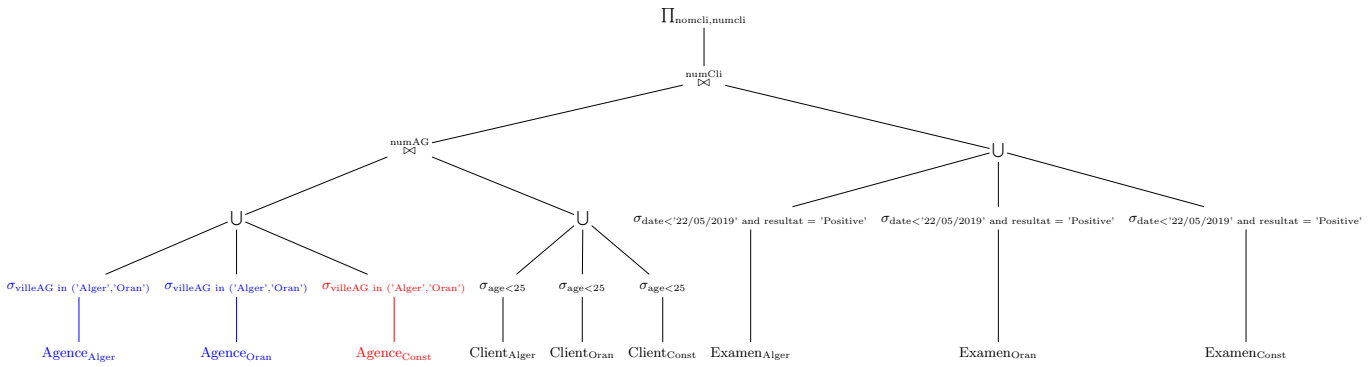
Arbre algebrique global :



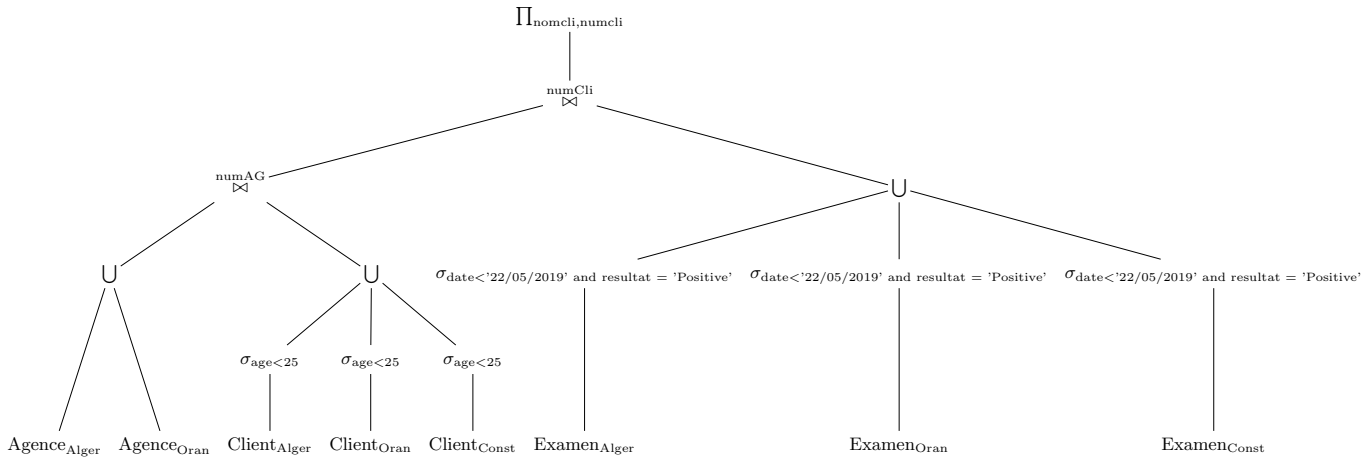

Arbre algebrique canonique (On remplace la table avec ces fragments et on utilise l'operateur de reconstruction $\bigcup$ ) :



regle optimisation generale (descente des operateurs de reductions de taille restriction avant $\bigcup$) :

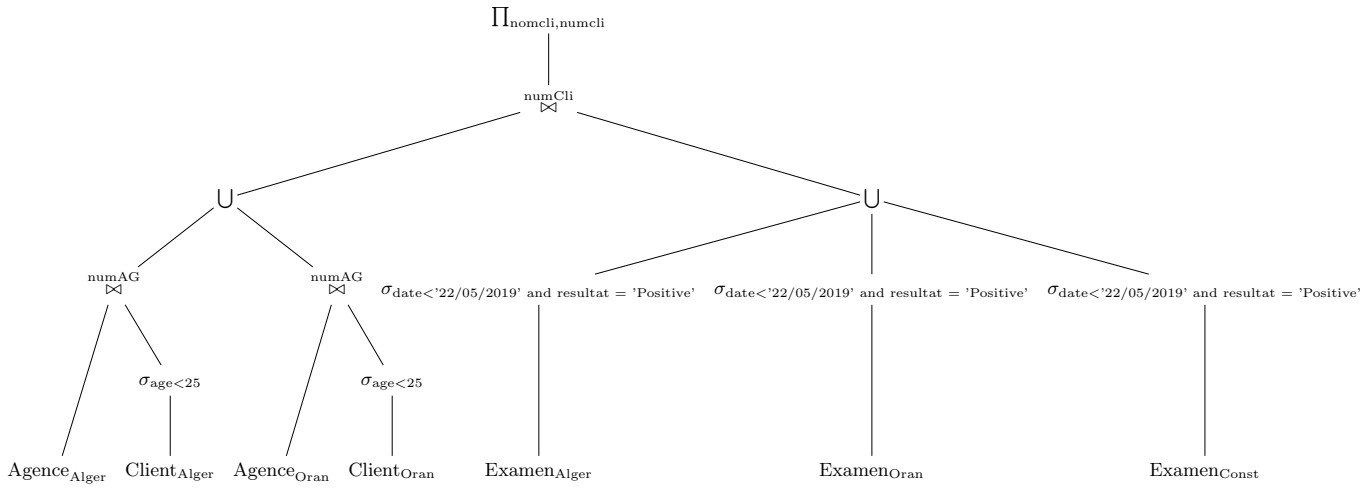



Suppression de fragment vide et restriction inutile:




Distribution de la jointure par rapport à l'union et élimination des fragments vides :
On met $\text{Agence}_{\text{Alger}} : A_1$ , $\text{Agence}_{\text{Oran}} : A_2$ , $\text{Client}_{\text{Alger}} : C_1$ , $\text{Client}_{\text{Oran}} : C_2$ , $\text{Client}_{\text{Const}} : C_3$ .

$$\begin{aligned}(A_1 \bigcup A_2) \bowtie (C_1 \bigcup C_2 \bigcup C_3) &= (A_1 \bowtie C_1) \bigcup \underbrace{(A_1 \bowtie C_2)}_{\emptyset} \bigcup \underbrace{(A_1 \bowtie C_3)}_{\emptyset} \bigcup \underbrace{(A_2 \bowtie C_1)}_{\emptyset} \bigcup (A_2 \bowtie C_2) \bigcup \underbrace{(A_2 \bowtie C_3)}_{\emptyset} \\ &= (A_1 \bowtie C_1) \bigcup (A_2 \bowtie C_2)\end{aligned}$$

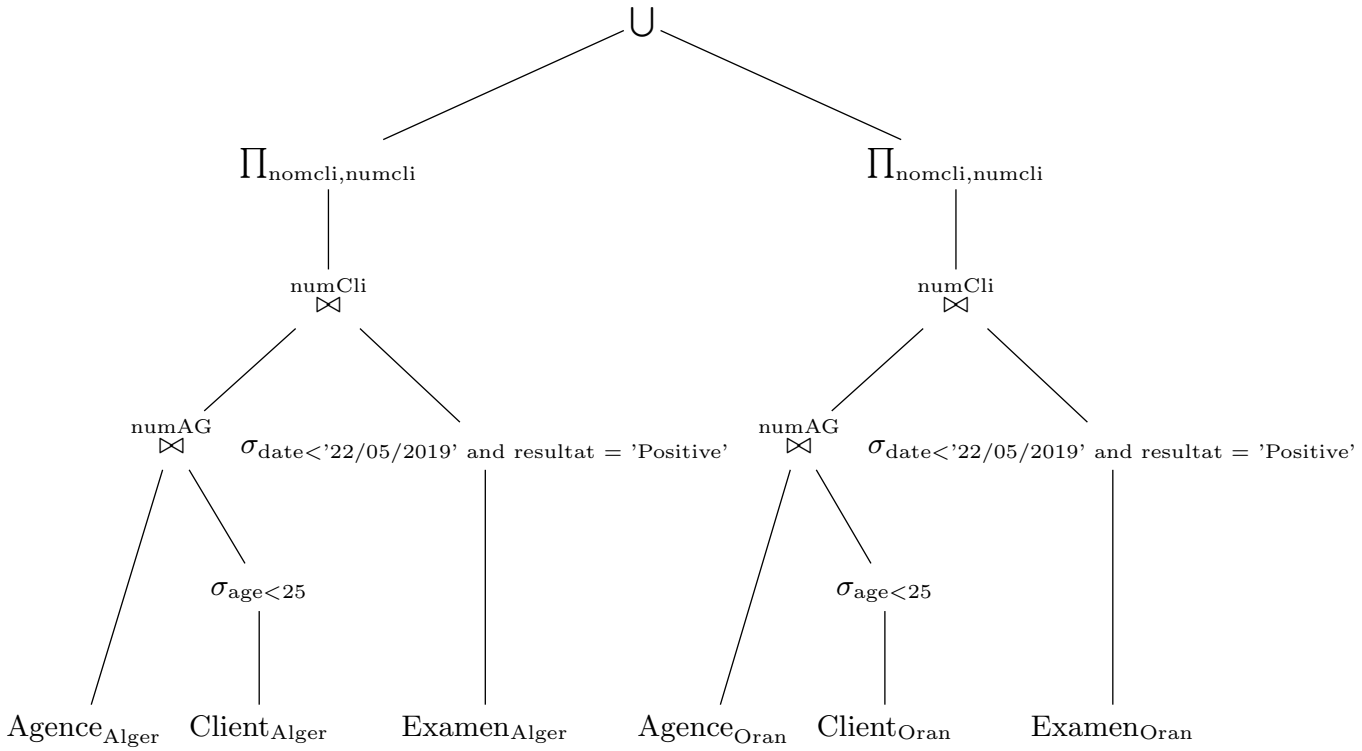


Distribution de la jointure par rapport à l'union et élimination des fragments vides, descente des operateurs de reductions de taille projection avant $\bigcup$ :

On met $(A_1 \bowtie C_1) : AC_1$ , $(A_2 \bowtie C_2) : AC_2$ , $\text{Examen}_{\text{Alger}} : E_1$ , $\text{Examen}_{\text{Oran}} : E_2$ , $\text{Examen}_{\text{Const}} : E_3$ .

$$(AC_1 \bigcup AC_2) \bowtie (E_1 \bigcup E_2 \bigcup E_3) = (AC_1 \bowtie E_1) \bigcup \underbrace{(AC_1 \bowtie E_2)}_{\emptyset} \bigcup \underbrace{(AC_1 \bowtie E_3)}_{\emptyset} \bigcup \underbrace{(AC_2 \bowtie E_1)}_{\emptyset} \bigcup (AC_2 \bowtie E_2) \bigcup \underbrace{(AC_2 \bowtie E_3)}_{\emptyset}$$

$$= (AC_1 \bowtie E_1) \bigcup (AC_2 \bowtie E_2)$$




- **Stratégie1** :Exécuter la partie gauche de l'arbre précédent à Alger Exécuter la partie droite à Oran et envoyer le résultat à Alger Faire l'union des deux résultats
- **Stratégie 2** :Envoyer les fragments Agence_Oran, Client_Oran et Examen_Oran à Alger Exécuter toute la requête sur Alger

## Exercice 3

On a

```sql
-- R1
SELECT NOM,PRENOM,EMAIL FROM UTILISATEUR WHERE DATE_DERNIER_ACCES < '01-03-2011';

-- R2
SELECT VILLE,PAYS,AGE FROM UTILISATEUR WHERE DATE_ENREGISTREMENT < '01-01-2011';

-- R3
SELECT NOM,PRENOM,VILLE FROM UTILISATEUR WHERE AGE > 18;
```

User1 aura besoin de l'attribut cle primaire IDU , les attributs selectioner dans la requete R1 (NOM,PRENOM,EMAIL) et l'attribut dans la condition where (DATE_DERNIER_ACCESS) :

$$\text{User1} = \prod_{\text{IDU,NOM,PRENOM,EMAIL,DATE_DERNIER_ACCESS}}(\text{Utilisateur})$$

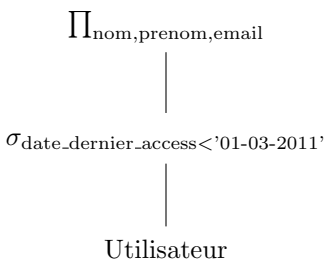
User2 aura besoin de l'attribut cle primaire IDU , les attributs selectioner dans la requete R2 (VILLE,PAYS,AGE) et l'attribut dans la condition where (DATE_ENREGISTREMENT) :

$$\text{User2} = \prod_{\text{IDU,VILLE,PAYS,AGE,DATE_ENREGISTREMENT}}(\text{Utilisateur})$$

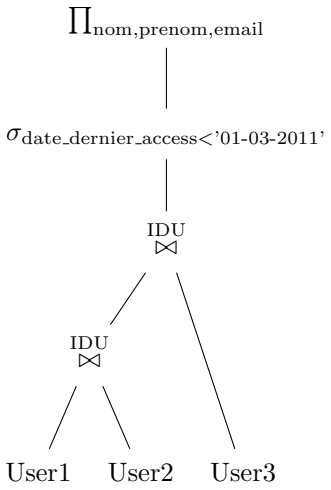
User3 aura besoin de l'attribut cle primaire IDU , les attributs restants (MOT_DE_PASSE,CODE_POSTAL,TELEPHONE):

$$\text{User3} = \prod_{\text{IDU,MOT_DE_PASSE,CODE_POSTAL,TELEPHONE}}(\text{Utilisateur})$$

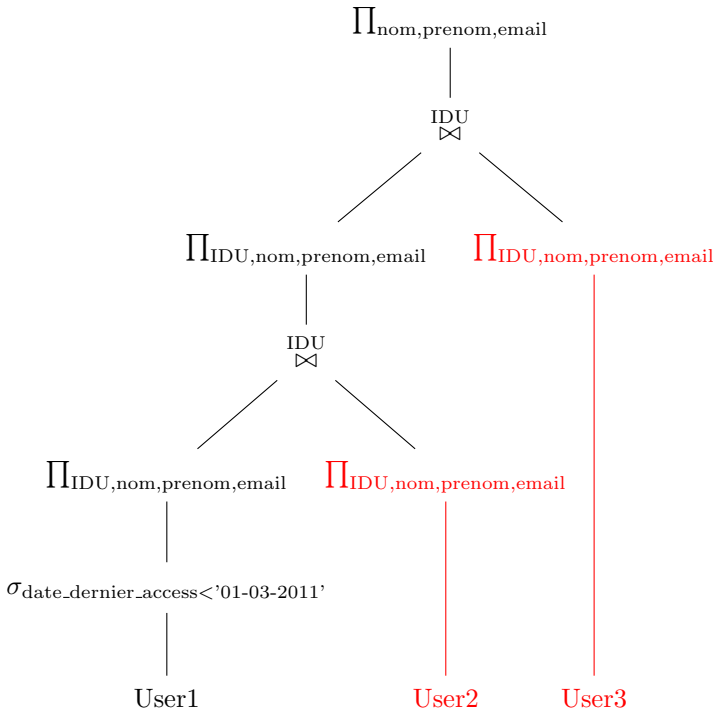
R1
Arbre algebrique global :



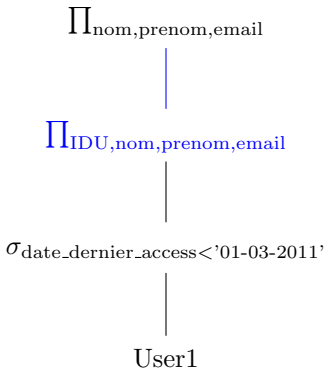

Arbre algebrique Canonique on remplace la table avec ces fragments et on utilise l'operateur de reconstruction $\bowtie$:



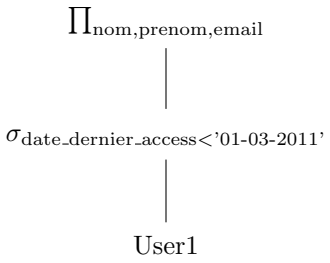
Optimisation General descente des operateur de reduction de taille projection, reduction :




Supression des fragments vide :



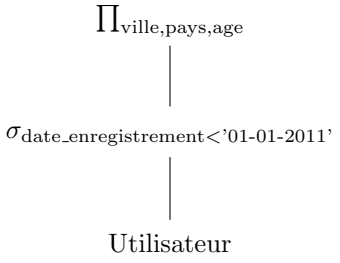

Supression projection inutile :



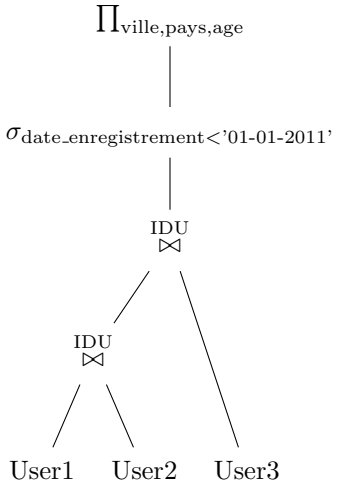

**R1**

```
SELECT NOM,PRENOM,EMAIL FROM USER1 WHERE DATE_DERNIER_ACCES < '01-03-2011';
```

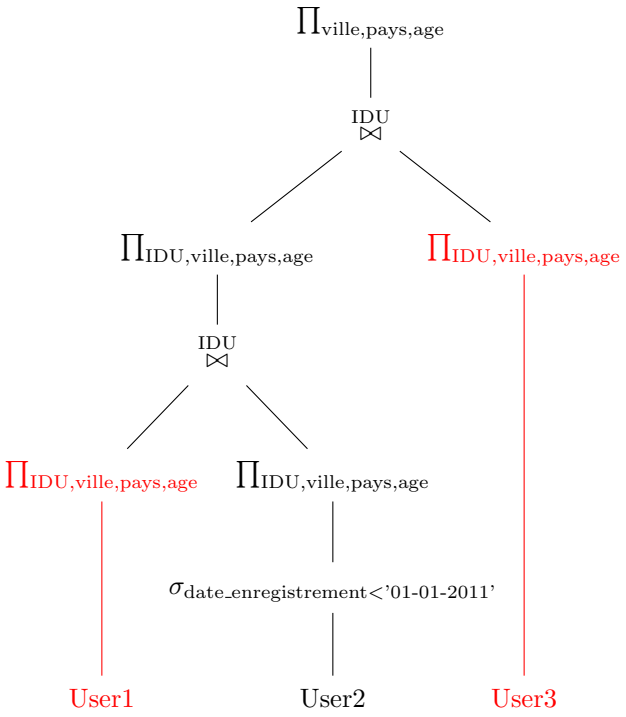
R2
Arbre algebrique global :



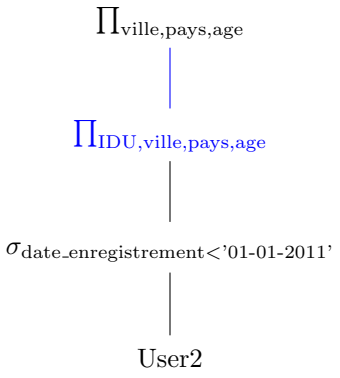

Arbre algebrique Canonique on remplace la table avec ces fragments et on utilise l'operateur de reconstruction $\bowtie$:



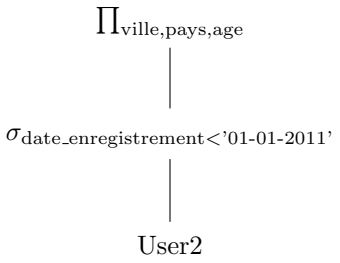

Optimisation General descente des operateur de reduction de taille projection, reduction :



Supression des fragments vide :



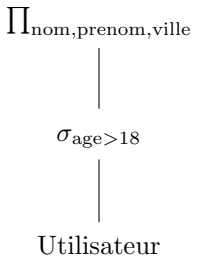

Supression projection inutile :



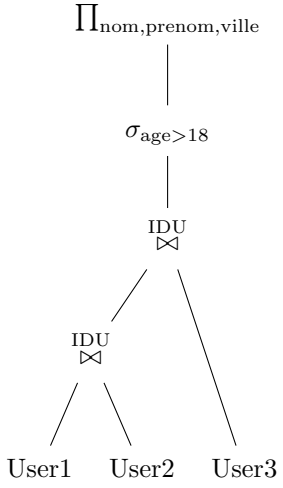

**R2**

```
SELECT VILLE,PAYS,AGE FROM USER2 WHERE DATE_ENREGISTREMENT < '01-01-2011';
```

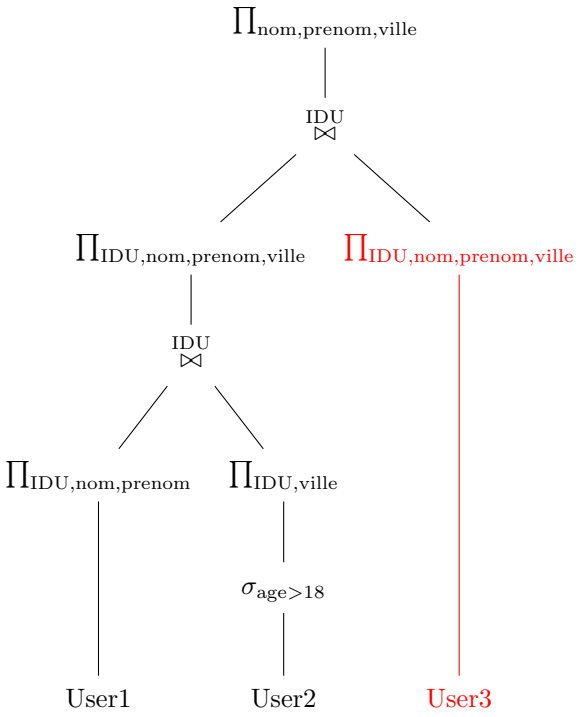
R3
Arbre algebrique global :



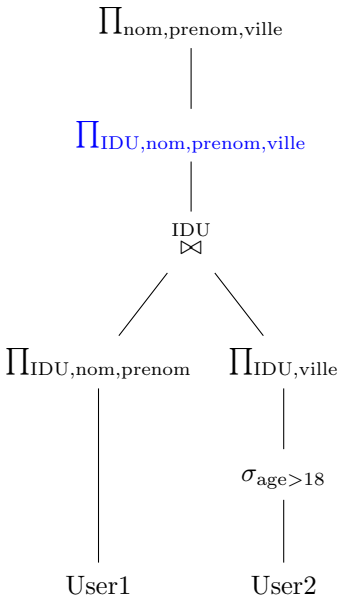

Arbre algebrique Canonique on remplace la table avec ces fragments et on utilise l'operateur de reconstruction $\bowtie$:



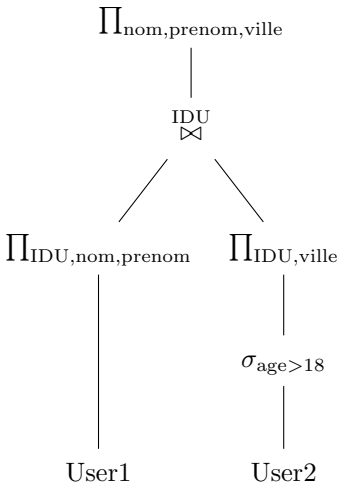
Optimisation General descente des operateur de reduction de taille projection, reduction :




Supression des fragments vide :



Supression projection inutile :



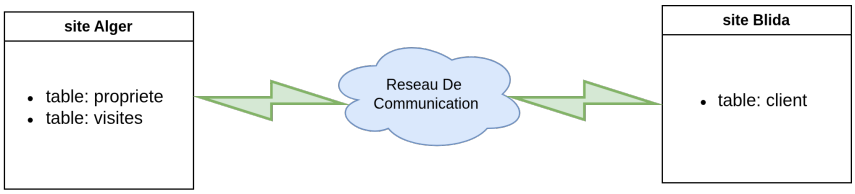

**R3**

```
SELECT NOM,PRENOM,VILLE FROM USER2,USER1 WHERE AGE > 18 AND USER1.IDU=USER2.IDU;
```

## Conclusion

Les deux requêtes R1 et R2 sont réécrites chacune sur un fragment car tous les attributs référencés par chacune des requêtes appartiennent au même fragment. La requête R3 accède à deux fragments, alors une opération de jointure supplémentaire entre USER1 et USER2 est ajoutée à la requête pour récupérer tous les attributs référencés

# Exercice 4




**Stratégie 1** : Déplacer la relation client à Alger et y traiter la requête.

$$\begin{aligned}
\text{temp}_{\text{envoie-Client}} &= \text{delai}_{\text{client}} + \frac{\text{taille} = \text{nb tuple}_{\text{client}} \cdot \text{ taille tuple}}{\text{taux-transmission}} \\
&= 1 + \frac{10^5 \cdot 10^2}{10^4} \\
&= 1 + 10^3 = 1001 \text{ s} \\
&= \frac{1001}{60} = 16.68 \text{ min}
\end{aligned}$$

**Stratégie 2** : Déplacer les relations propriété et visite à Blida et y traiter la requête

$$\begin{aligned}
\text{temp}_{\text{envoie-Propriete}} + \text{temp}_{\text{envoie-Visites}} &= \text{delai}_{\text{Propriete}} + \frac{\text{nb tuple}_{\text{propriete}} \cdot \text{taille tuple}}{\text{taux-transmission}} + \text{delai}_{\text{visites}} + \frac{\text{nb tuple}_{\text{visites}} \cdot \text{taille tuple}}{\text{taux-transmission}} \\
&= 1 + \frac{10^4 \cdot 10^2}{10^4} + 1 + \frac{10^6 \cdot 10^2}{10^4} \\
&= 2 + 10^2 + 10^4 = 10102 \text{ s} \\
&= \frac{10102}{3600} = 2.8 \text{ h}
\end{aligned}$$

**Stratégie 3** : joindre les deux relations propriété et visite à Alger et pour chaque tuple du résultat vérifier à Blida si le prix max est supérieur à 30 000. La vérification de chaque tuple suppose deux messages : l'envoi d'un tuple et une réponse

$$\begin{aligned}
\text{temp}_{\text{envoie-joinTuple}} + \text{temp}_{\text{envoie-reponse}} &= \text{delai}_{\text{joinTuple}} + \frac{\text{nb tuple}_{\text{join}} \cdot \text{taille tuple}}{\text{taux-transmission}} + \text{delai}_{\text{reponse}} + \frac{\text{taille 1 bit}}{\text{taux-transmission}} \\
&= 1 \cdot 10^5 + \frac{10^5 \cdot 10^2}{10^4} + 1 \cdot 10^5 + \frac{\frac{1}{8}}{10^4} \\
&= 201000.0000125 \text{ s} \\
&= \frac{\frac{201000.0000125}{3600}}{24} = 2.3 \text{ j}
\end{aligned}$$

**Stratégie 4** : sélectionner les clients de prix max est supérieur à 30 000 à Blida et pour chaque tuple du résultat vérifier à Alger s'il y a une propriété visitée

$$\begin{aligned}
\text{temp}_{\text{envoie-clientTuple}} + \text{temp}_{\text{envoie-reponse}} &= \text{delai}_{\text{clientTuple}} + \frac{\text{nb tuple}_{\text{client(restriction)}} \cdot \text{taille tuple}}{\text{taux-transmission}} + \text{delai}_{\text{reponse}} + \frac{\text{taille 1 bit}}{\text{taux-transmission}} \\
&= 1 \cdot 10 + \frac{10 \cdot 10^2}{10^4} + 1 \cdot 10 + \frac{\frac{1}{8}}{10^4} \\
&= 20.1 \text{ s}
\end{aligned}$$

**Stratégie 5** : joindre les relations propriété et visite à Alger . Projeter le résultat sur numéro propriété et numéro client et déplacer ce dernier à Blida pour les tuples ayant le prix max supérieur à 30 000. Pour simplifier, nous suivons l'hypothèse que le résultat de la projection est long également de 100 caractères.

$$\begin{aligned}
\text{temp}_{\text{envoie-join}} &= \text{delai}_{\text{join}} + \frac{\text{nb tuple}_{\text{join}} \cdot \text{ taille tuple}}{\text{taux-transmission}} \\
&= 1 + \frac{10^5 \cdot 10^2}{10^4} \\
&= 1 + 10^3 = 1001 \text{ s} \\
&= \frac{1001}{60} = 16.68 \text{ min}
\end{aligned}$$

**Stratégie 6** : sélectionner les clients de prix max est supérieur à 30 000 à Blida et déplacer le résultat à Alger pour y exécuter la requête.

$$\begin{aligned}
\text{temp}_{\text{envoie-clientRestrie}} &= \text{delai}_{\text{join}} + \frac{\text{nb tuple}_{\text{clientRestrie}} \cdot \text{ taille tuple}}{\text{taux-transmission}} \\
&= 1 + \frac{10 \cdot 10^2}{10^4} \\
&= 1 + 10^{-1} = 1.1 \text{ s}
\end{aligned}$$